# 03 Kontrollfluss und Bedingungen

## Kontrollfluss und Bedingungen

**Vorlesung 19.10.2026**
Quelle: 03_Kontrollfluss und Bedingungen.pdf

### Kontrollfluss

**Vorlesung 19.10.2026**
Themen: Programmablauf und Kontrollstrukturen · Formulierung und Auswertung von Bedingungen · Varianten der if-Anweisung (04 Kontrollstrukturen)

#### Der Kontrollfluss

- Ein Programm besteht aus einer beliebigen Kombination von **einfachen** und **komplexen** Anweisungen (Kontrollstrukturen).
- **Kontrollfluss** = Ablauf des Programms, bestimmt durch die **Reihenfolge**, in der Anweisungen abgearbeitet werden. Bei der Programmierung legt man ihn fest.

| Art | Beispiele |
|---|---|
| **Einfache Anweisung** | Wertzuweisung, Ausgabe auf der Konsole |
| **Komplexe Anweisung** | bedingte Anweisung (`if`), `while`-Schleife, `for`-Schleife, `switch` |

**Einfache Anweisungen** Werden **der Reihe nach (sequenziell)** vollständig abgearbeitet. Beispiele: Variable definieren, Wert zuweisen, initialisieren, einzelner Verarbeitungsschritt (z.B. Ausgabe mit `printf`).

```
initialisiere eine int Variable a mit einem Wert
initialisiere eine int Variable b mit einem Wert
addiere a zu b und speichere das Ergebnis in c
gib c auf der Console aus.
```

**Bedingte Anweisung (`if`)** Eine Gruppe von Anweisungen wird **abhängig von einer Bedingung** ausgeführt → im Kontrollfluss entstehen **Verzweigungen**.

```
falls die Variable a den Wert 0 hat
  dann gib den Wert von b zurück
andernfalls
  berechne b - a und weise das Ergebnis der Variable a zu.
```

**Schleifen** Eine Gruppe von Anweisungen wird so oft ausgeführt, bis eine Bedingung **nicht mehr** gilt.

```
solange b größer 0 ist
  ziehe a von b ab und weise das Ergebnis b zu.
```

### Kombination von Kontrollstrukturen

- Anweisungen und Kontrollstrukturen können **beliebig kombiniert und geschachtelt** werden.
- Bedingte Anweisungen und Schleifen dürfen sich jedoch **nicht überschneiden**.
- Grenzen werden mit **geschweiften Klammern** `{ }` markiert. Sie sind Teil der Syntax und dürfen (i.d.R.) nicht weggelassen werden (**Compiler-Fehler**).
- Geschachtelte Strukturen werden durch **Einrückung** verdeutlicht (**Coding Convention**).

### Beispiel (Pseudocode): Was wird berechnet?

```
initialisiere eine Variable a vom Typ int
initialisiere eine Variable b vom Typ int
falls a gleich 0 {
    dann gib b zurück
} andernfalls {
    solange b ungleich 0 {
        falls a größer b {
            weise a den Wert von a - b zu
        } andernfalls {
            weise b den Wert von b - a zu
        }
    }
}
gib a zurück
```

> **Auflösung**
> Das ist der **Euklidische Algorithmus** (Subtraktionsvariante): berechnet den **größten gemeinsamen Teiler (ggT)** von a und b. Der Fall `a == 0` liefert direkt `b`. (Die Folie verrät die Lösung nicht, dies ist meine Einordnung.)

### Bedingungen im Kontrollfluss

- Eine **Bedingung** ist ein **Ausdruck**, dessen Auswertung immer einen **Wahrheitswert** (`true`/`false`) liefert.
- Eine Bedingung ist **keine Anweisung**, kann also nicht für sich allein stehen.
- Sie kann in einer einfachen Anweisung einer `boolean`-Variable zugewiesen werden:

```
int x = …;              // lies Wert für x von der Kommandozeile ein
boolean b = x > 0;
```

- Vor allem nötig für **bedingte Anweisungen** und **Schleifen**: Bedingung im `if` erlaubt unterschiedliche Abläufe; in der `while`-Schleife führt sie zur Beendigung, sobald sie nicht mehr wahr ist.

Operatoren zur Formulierung:

**Zusammenfassung**

- Der Kontrollfluss entspricht dem **Algorithmus**, der umgesetzt werden soll; er wird über Kontrollstrukturen im Code nachgebildet.
- Unterscheide: einfache Anweisungen, bedingte Anweisungen, Schleifen und Switches.
- Der Ablauf wird in vielen Kontrollstrukturen über **Bedingungen** gesteuert.
- Zur Formulierung: **arithmetische, relationale und logische Operatoren**; die Regeln zur Auswertungsreihenfolge sind zu beachten.

## Operatoren in Bedingungen

Bedingungen sind Ausdrücke aus **konkreten Werten, Variablen und Operatoren**. Es gibt drei Operatorarten:

| Art | Operanden | Ergebnis |
|---|---|---|
| **arithmetisch** | numerisch | numerisch |
| **relational** | numerisch | Wahrheitswert |
| **logisch** | Wahrheitswerte | Wahrheitswert |

**Auswertungsreihenfolge einer Bedingung**

1. erst **arithmetische** Ausdrücke,
2. dann **relationale** Ausdrücke,
3. dann **logische** Ausdrücke.

### Arithmetische Operatoren

(Details und Prioritäten: 02 Variablen und Datentypen)

| Symbol | Assoziativität | Bedeutung |
|---|---|---|
| `+` | rechts | positives Vorzeichen |
| `-` | rechts | negatives Vorzeichen |
| `+` | links | Addition |
| `-` | links | Subtraktion |
| `*` | links | Multiplikation |
| `/` | links | Division |
| `%` | links | Modulo |

### Relationale Operatoren

| Symbol | Assoziativität | Bedeutung |
|---|---|---|
| `>` | links | größer |
| `<` | links | kleiner |
| `>=` | links | größer gleich |
| `<=` | links | kleiner gleich |
| `==` | links | gleich |
| `!=` | links | ungleich |

### Logische Operatoren

| Symbol | Assoziativität | Bedeutung |
|---|---|---|
| `!` | rechts | **Verneinung**: aus `true` wird `false` und umgekehrt |
| `&&` | links | **UND**: `true`, wenn beide Operanden `true`. Links wird zuerst ausgewertet; nur wenn dieser `true` ist, wird auch rechts ausgewertet (**Short Circuit**). `&&` hat **höhere Priorität** als `\|\|` |
| `&` | links | UND bei `boolean`-Operanden; es werden **immer beide** Operanden ausgewertet |
| `\|\|` | links | **ODER**: `true`, wenn links oder rechts oder beide wahr. Rechts wird nur ausgewertet, wenn links `false` ist (**Short Circuit**) |
| `\|` | links | ODER bei `boolean`-Operanden; **immer beide** Operanden ausgewertet |
| `^` | links | **exklusives ODER (XOR)**: entweder der erste oder der zweite Operand ist wahr, nicht beide gleichzeitig; sind beide gleich, ergibt es `false` |

### Wahrheitstabellen

| `a` | `b` | `a && b` / `a & b` | `a \|\| b` / `a \| b` | `a ^ b` |
|---|---|---|---|---|
| true | true | true | true | **false** |
| true | false | false | true | true |
| false | true | false | true | true |
| false | false | false | false | false |

Negation: `!true` `false`, `!false` `true`.

### Unvollständige Auswertung (Short Circuit)

Warum gibt es `&&`/`&` und `||`/`|` jeweils doppelt?
1. **Optimierung**: der zweite Operand muss in manchen Fällen nicht mehr ausgewertet werden.
2. **Problemfall**: die Auswertung des rechten Ausdrucks **hängt vom Ergebnis des linken** ab.

```
b != 0 && a / b > 0   // Auswertung bricht ab, falls links false
b != 0 &  a / b > 0   // bricht NICHT ab => Fehler (Division durch 0)
```

Übungen dazu: und

## Übungsaufgaben

**Lösungen sind eingeklappt (Pfeil anklicken). Wo die Folie keine Lösung zeigt, ist es meine eigene, bitte gegenprüfen.**

### Übung Bedingungen formulieren

Mit `int i, j, k` und `boolean a, b, c`: ()

| Text | Java |
|---|---|
| i, j und k sind alle verschieden von 0 | ? |
| i ist durch 17 teilbar und echt positiv | ? |
| j ist ungerade und liegt echt zwischen 20 und 40 | ? |
| k ist 0 und es gilt nicht a | ? |
| es gilt genau a oder genau b | ? |
| es gilt nicht c | ? |
| es gilt b oder nicht c, und immer gilt i ist echt kleiner 0 | ? |

**Lösung (eigene)**

```
i != 0 && j != 0 && k != 0
i % 17 == 0 && i > 0
j % 2 != 0 && j > 20 && j < 40
k == 0 && !a
a ^ b
!c
(b || !c) && i < 0
```

### Übung Bedingungen prüfen

Korrekt oder nicht? Mit `int i, j, k` und `boolean a, b`:

1. `i && (j || k)`
2. `i % 32 || j > 23 * 5 && !b`
3. `i*j*k=0`
4. `i % 4 == 0 && i % 100 > 0`
5. `i < j && a && b`
6. `a * b == 0`
7. `!(a > b && i < j)`

**Lösung (eigene, mehrere Korrekturen möglich)**

1. `&&`/`||` brauchen `boolean`, nicht `int` → `i != 0 && (j != 0 || k != 0)`
2. `i % 32` ist `int` → `i % 32 != 0 || j > 23 * 5 && !b`
3. `=` ist Zuweisung, nicht Vergleich → `i * j * k == 0`
4. 
5. 
6. Arithmetik mit `boolean` nicht erlaubt → `!(a && b)`
7. relationale Operatoren (`>`) brauchen numerische Operanden → z.B. `!(a && !b && i < j)`